# Softwaredesign
# Website tabicat.co.uk

Jan Merz, Celina Riegel, Jasmin Steidl, Laura van Uffelt

March 5, 2026

## 1 Einleitung

Dieses Dokument beschreibt das Softwaredesign der Website *tabicat.co.uk*. Das Design basiert auf der umzusetzenden Architektur (statische Website + minimale Server-Komponente) und fokussiert sich auf das **Booking-Formular**, da es der komplexeste und kritischste Teil des Systems ist. Unter anderem muss dieser Teil des Systems mit mehreren Personen, einer temporären Datenspeicherung im Browser und dem Versand der Informationen per Mail umgehen. Zusätzlich enthält das Dokument einen strukturierten Überblick über zentrale Designentscheidungen sowie schematische UML Darstellungen.

## 2 Überblick über das Softwaredesign

### 2.1 Designziele

Das Design soll die wichtigsten funktionalen und nicht-funktionalen Anforderungen erfüllen, bei gleichzeitig möglichst geringer Systemkomplexität.

- **Wartbarkeit:** Inhalte der Website sollen durch die Anbieterin über Hugo-Markdown-Dateien (z.B. via GitHub) angepasst werden können.
- **Minimales Backend:** Serverseitige Logik nur dort, wo sie notwendig ist (Formularverarbeitung und E-Mail Versand).
- **Datenschutzorientierung:** Keine persistente Speicherung personenbezogener Daten (keine Datenbank); Verarbeitung nur zum Erstellen und Versenden einer E-Mail.
- **Zuverlässigkeit:** Booking-Anfragen müssen die Anbieterin per E-Mail erreichen.

### 2.2 Umfang

Der statische Teil der Website umfasst u.a. Home, Weekend Trips, Voyagers, Visitors, Customised Trips und Contact. Dieses Dokument konzentriert sich auf den **Booking-Formular-Flow**. Das Contact-Formular ist konzeptionell eine vereinfachte Form des Booking-Formulars und wird daher nicht im Detail beschrieben.

## 3 Designstruktur

### 3.1 Komponentensicht

Das System ist in die in fig. 1 dargestellten Komponenten zerlegt. Eine zentrale Designentscheidung ist die Trennung zwischen der statischen Website (Hugo auf GitHub Pages) und einem minimalen PHP-Service (gehostet über InfinityFree), der ausschließlich für Formular-Submissions genutzt wird und GitHub Pages nur statische HTML Seiten erlaubt.

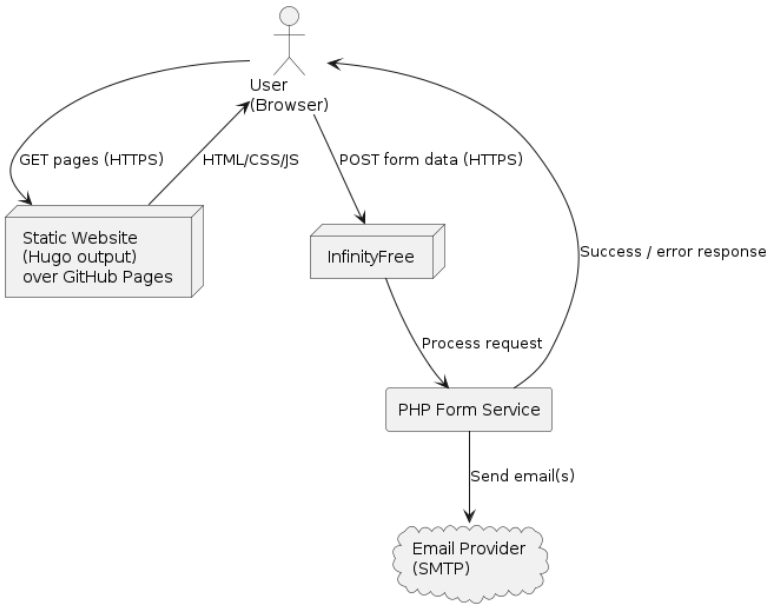


Figure 1: Komponentensicht (statische Hugo-Website + PHP Form-Service)

## 3.2 Verantwortlichkeiten

| Komponente | Aufgaben / Verantwortlichkeiten |
|---|---|
| **Browser (Client)** | • Darstellung der statischen Seiten<br>• Anzeige des Booking-Formulars<br>• Multi-Person-Flow<br>• Temporäre Speicherung eingegebener Daten<br>• Client-seitige Validierung |
| **PHP Form-Service (Server)** | • Empfang der Booking-Daten als JSON<br>• Serverseitige Validierung und Sanitizing<br>• Erzeugung des E-Mail Inhalts<br>• Versand der E-Mail über einen E-Mail Provider<br>• Rückgabe einer JSON-Antwort |
| **Hugo Content (Markdown)** | • Inhalte als Markdown-Dateien<br>• Anpassung der Inhalte durch die Anbieterin via GitHub<br>• Generierung statischer HTML-Seiten für das Deployment |

Table 1: Verantwortlichkeiten der zentralen Komponenten

# 4 Schnittstellendesign

## 4.1 Booking-Submission Endpoint

Das Booking-Formular wird über einen einzelnen Endpoint verarbeitet, der auf InfinityFree gehostet ist.

- **Endpoint:** `send_email.php`
- **Methode:** POST
- **Content-Type:** `application/json`
- **Request Body:** JSON-Objekt mit `formInfo` (Array von Personen) und `tripInfo`
- **Response Body:** JSON {`success`: boolean, `message`: string}

## 4.2 Datenverarbeitung und Datenschutzgrenze

Das System ist so entworfen, dass die Backend-Komplexität minimal bleibt und keine persistente Speicherung notwendig ist:

- Booking-Daten werden während der Formulareingabe temporär im Browser gespeichert.
- Der PHP-Service nutzt die erhaltenen Daten ausschließlich, um die Booking-E-Mail zu generieren und zu versenden.
- Es wird keine Datenbank eingesetzt; der Server speichert Booking-Daten nicht dauerhaft.

# 5 Verhaltensdesign (Booking-Workflow)

## 5.1 Beschreibung des Booking-Workflows

Das Booking-Formular unterstützt mehrere Reisende. Der Workflow ist als einzelnes Formular umgesetzt, das für jede Person wiederverwendet wird. Beim Wechseln zwischen Personen werden die Eingaben temporär im LocalStorage abgelegt. Beim Bestätigen wird über alle gespeicherten Personen iteriert und validiert. Wenn die Validierung erfolgreich ist, werden alle Booking-Daten als JSON an den PHP-Service gesendet, der die Booking-E-Mail verschickt. Bei erfolgreicher Verarbeitung werden booking-relevante Einträge im LocalStorage gelöscht und der Client auf eine `thanks`-Seite weitergeleitet.

## 5.2 Sequenzdiagramm

Die Interaktion ist in fig. 2 dargestellt.

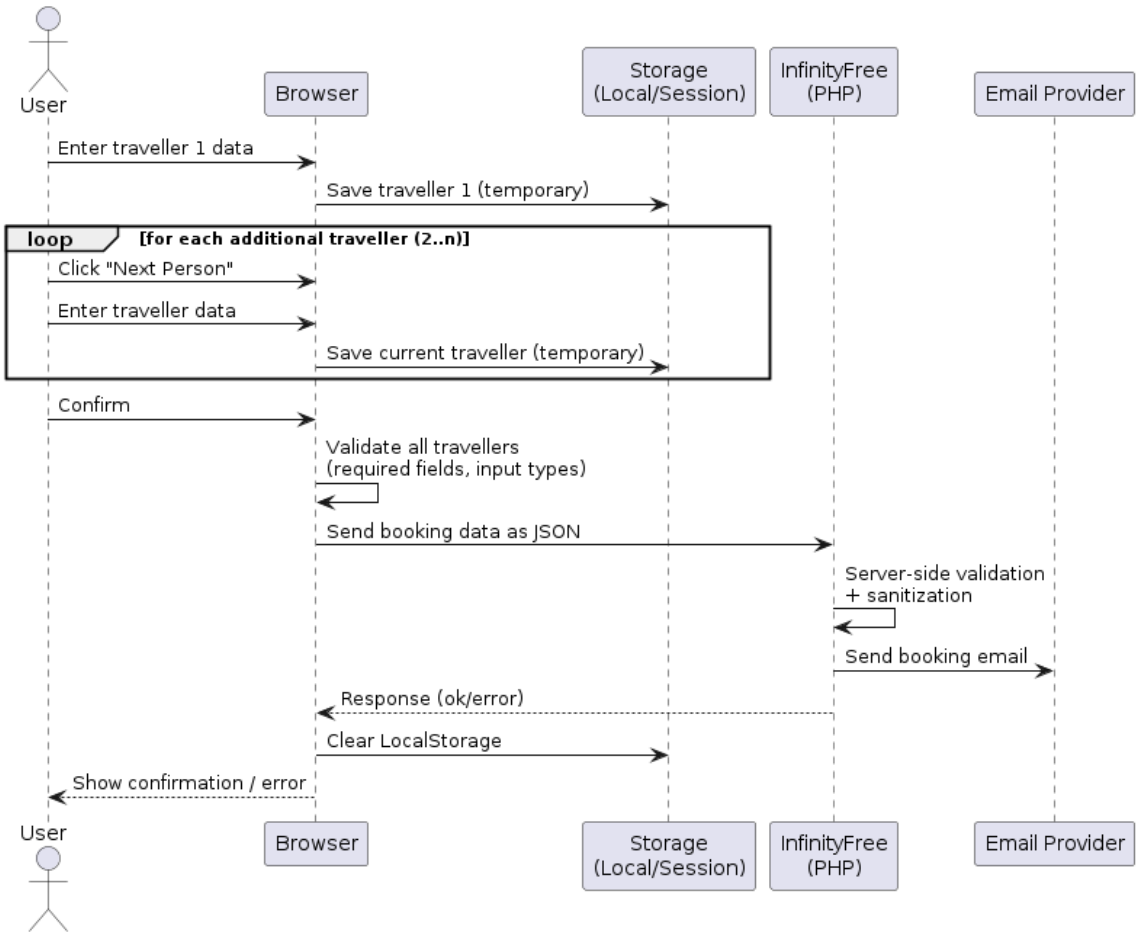


Figure 2: Sequenzdiagramm des Booking-Submission-Flows

### 5.3 Validierungsstrategie

Die Validierung ist bewusst zweistufig:

- **Client-seitig:** Prüfung ob Pflichtfelder ausgefüllt wurden, vor es möglich ist das Formular abzuschicken
- **Server-seitig:** PHP prüft Preisberechnung und sanitisiert Textinhalte für sichere E-Mail Generierung.

Das ist notwendig, da client-seitige Validierung umgangen werden kann.

## 6 Daten-Design

### 6.1 Client-seitige Storage Keys

Das Booking-Formular nutzt Browser-Storage zur Unterstützung der Multi-Person Eingabe:

- **SessionStorage:** `currentPerson` (Index der aktuell angezeigten Person)
- **LocalStorage:** `formInfo` (Enthält eingegebene Informationen aller Reisenden), `tripInfo` (allgemeine Informationen darüber welcher Ausflug gewählt wurde), `numberOfPersons`, `studentTrip` (Boolean)

### 6.2 Storage-Lifecycle

Um das Risiko zu reduzieren, dass personenbezogene Daten unnötig im Browser verbleiben, gilt folgende Designregel:

- Bei erfolgreichem E-Mail Versand werden alle buchungsrelevanten Daten aus dem Local-Storage gelöscht.

## 7 Zentrale Designentscheidungen

### 7.1 Static-first Contentpflege ("Admin light")

Ein vollständiger Admin-Bereich mit Login wurde als zu aufwendig für den Projektumfang bewertet. Stattdessen wird Hugo + Markdown verwendet, sodass die Anbieterin Inhalte über GitHub pflegen kann. Das reduziert Projektrisiken und erhöht die Wartbarkeit.

### 7.2 Minimales Backend (E-Mail only)

Das Backend ist bewusst auf Formularverarbeitung und E-Mail Versand begrenzt. Dadurch bleibt die Systemkomplexität gering und es ist keine dauerhafte Speicherung personenbezogener Daten notwendig.

## 8 Risiken und Maßnahmen

- **Sensible Booking-Daten:** Das Booking-Formular erfasst personenbezogene und teilweise sensible Daten (z.B. Passnummer, gesundheitliche Hinweise).
  *Maßnahmen:* keine Server-Persistenz; HTTPS; zweistufige Validierung; Storage-Cleanup nach Erfolg; Datensparsamkeit prüfen.

- **Zuverlässigkeit des E-Mail Versands:**
  *Maßnahmen:* klare Fehlermeldungen an den Client; robuste Mail-Konfiguration als mögliche Verbesserung.

- **Verbleibende Daten im Browser:**
  *Maßnahmen:* explizites Löschen der LocalStorage Keys bei Erfolg; keine Speicherung über das Notwendige hinaus.